School of Computer Science, McGill University

COMP-421 Database Systems, Winter 2025*

Written Assignment Description 3: Graph Databases and Query Evaluation

Due Date April-2, 12:00 NOON

There are a total of 80 points on this assignment!

Note that some questions below have zero points. You do not need to submit them for the assignment but they might be useful for general practice and preparation for the final exam.

Provide your solution in a single file. This is an individual assignment.

Part I: Graph Databases

Ex. 1 — Modeling Graph Databases vs. E/R (15 points)

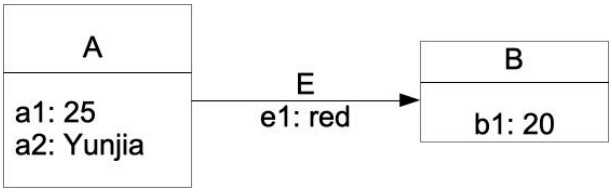
Given below information about politicians. Each politician is identified by their name, might have an address and a website and can be friends with other politicians. A politician can be a mayor of a city or a member of parliament for a given riding. Cities and ridings belong to provinces.

- Politicians have a name (non-unique) and might have a website. Some are currently mayors or members of parliament (federal) or have been in the past.
- A city has a name (non-unique), a population and belongs to a province (unique).
- A political riding (for federal elections) has a name (unique) and a population and belongs to a province.
- We keep track of the current and past mayors of each city, and which year they started and which year they ended to be mayor. The current mayor can be identified by not having an end year.
- Similarly, we keep track of the current and past members of parliament for each riding, again with start and end year.
- Politicians can be friends with each other.

1. (15 points) *With this information, draw an appropriate graph database that contains the following information. Plante* has been the current mayor of the city of Montreal (Quebec, population 1,800,000) since 2017. Her webpage is https://montreal.ca/en/elected-officials/valerie-plante-1211. From 2013-2017, Coderre was mayor of Montreal. Coderre was also Member of Parliament (MoP) for the riding Bourassa (Quebec, population of 105,600) from 1997-2013.

After him, Dubourg became MoP for Bourassa, and has remained in this position since then. Plante and Dubourg are friends.

Use the Neo4J graph model with nodes that have types/labels and properties, and directed relationships between the nodes that also have types and properties. Follow the notation as illustrated in the example graph below. There is one node with label A with properties a1 and a2 and one node with label B with property B1. There is an edge of type/label E with property e1. You DO NOT need to write Cypher statements to setup your data.



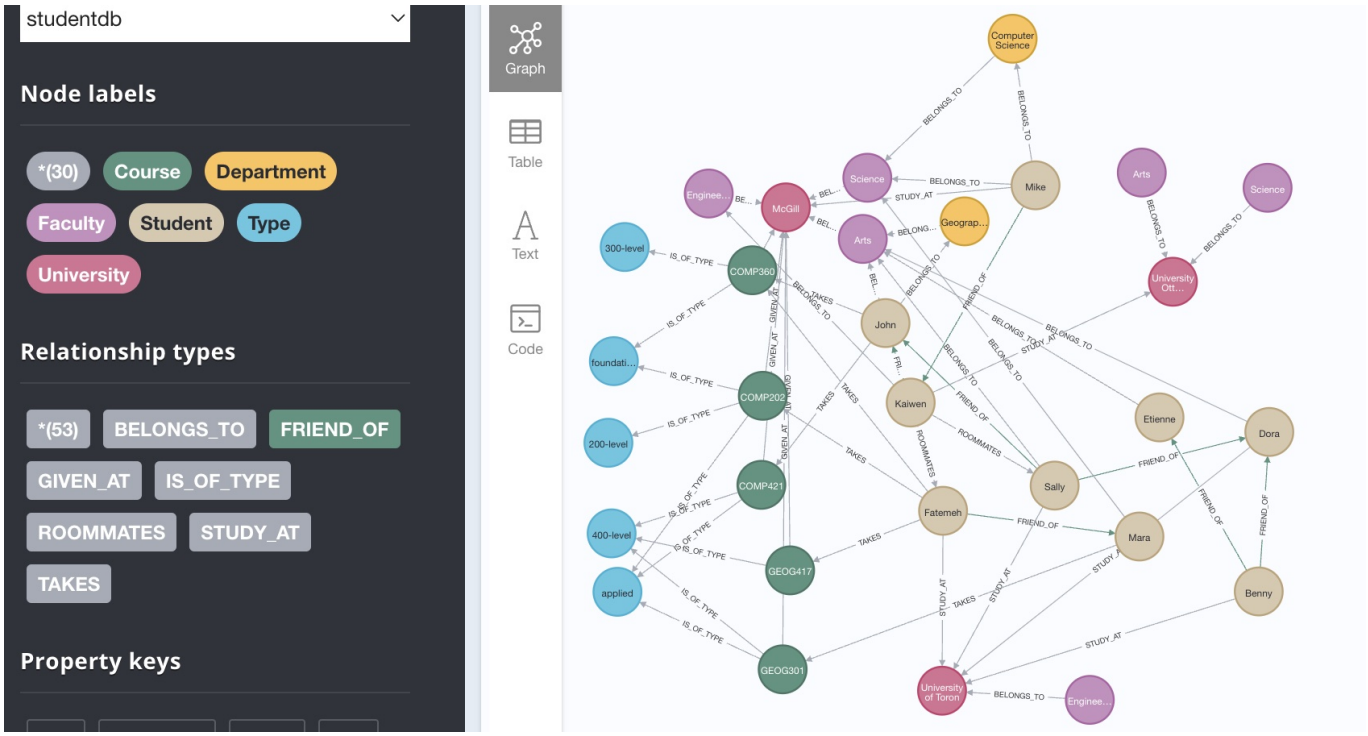
2. (0 points) Now provide a Cypher query that gives me the names of politicians who were mayor of Montreal twice. That is, they were mayor of Montreal, followed by one or more other mayors of Montreal, and then they became mayor again (e.g., Jean Drapeau was mayor of Montreal from 1954-1957 and then again from 1960-1986 while Sarto Fournier was mayor from 1957-1960). Again your query should be correct not only on your small example graph but for larger graphs that follow the same design schema but with more politicians, cities, ridings, etc.
3. (0 points) How would you model the same information in an E/R schema? Only indicate the entity sets, the relationship sets and the primary keys. This question is not graded but you are encouraged to answer it and then compare your solution with the example solution for a better understanding of graph databases and how they can express certain schema constraints.

Ex. 2 — Cypher Queries (15 points)

For this question, you can simply write your queries into your solution pdf.

Feel free to first try them out in Neo4j. For that, you can download Neo4j Desktop onto your computer, and use the script in the Appendix to create your database.

Below graph shows a student database with students, universities, faculties, departments and courses. Courses are given at a specific university and can be of certain types, faculties belong to a university and departments belong to a faculty. Students study at a university, might belong to a faculty and a department, and take courses. Students can also be friends of each other. You can find a textual description of the graph in the Appendix. Note that while the graph shows directed edges for friends and roommates (as all edges in Neo4j must be directed), they should be used with a semantics that the relationship is symmetric (if A is friend/roommate of B, then B is friend/roommate of A). Note that the database is not complete (some students don't study at any university, only McGill has courses, ...).



Given this graph database, answer the following question using Cypher queries

1. (0 points) Find all 300-level courses.
2. (0 points) Find all pairs of students that are either friends or roommates.
3. (0 points) Find all pairs of students that are roommates but not friends
4. (5 points) Find all pairs of students that are friends and roommates.
5. (5 Points) Find the names of the students who have taken courses at a different university than the one they study at.
6. (5 Points) Find pairs of students that take the same course and have a linkage through friend relationships (i.e., there is a path of edges labelled FRIEND_OF that connects those two students).

Part II: Query Evaluation

In this assignment, we evaluate queries for a banking - credit card database. We look at three relations `Account`, `Card`, and `Payment`, listed below.

Account (accountid:INT, name:VARCHAR(30), email:VARCHAR(50), address:VARCHAR(60), balance:INT , accounttype:VARCHAR(12))
9012101, 'Joe Rich', joe.rich@maildomain.com '1180 Rue. St. Mathieu, Montreal', 2500.0, 'checking'

Card (cardid:INT, owneraccountid:INT, paymentdue:INT, cardtype:CHAR(7))
$\longrightarrow$ owneraccountid references Account.
122202, 9012101, 250.0, 'AIRPLAN'

Payment (paymentid:INT, srcaccountid:INT, cardid:INT amount:INT, paydate:CHAR(10))
$\longrightarrow$ srcaccountid references Account, cardid references Card.
6223451, 9012101, 122202, 205.0, '2025-03-05'

INT has 4 Bytes, a char has 1 Byte. Assume for each VARCHAR attribute in the Account table, the average size of a value is half the total capacity allocated for that attribute. (e.g: `name` is on an average 15 characters).

There are 8,000 members in the bank, each with one account (no joint accounts, etc.) and 10,000 records in the `Card` relation .

Payments are made on an average once per month per card. The database has 1 full year of data stored in it (12 months - 365 days). As such, there are 120,000 payments stored in the `Payments` relation. A payment can be made from an account that is not necessarily the account attached to the card. (ie., `srcaccountid` associated with a particular payment of a card need not be the `owneraccountid` associated with the card itself. As in, someone else paid off the bill).

`Account balance` and `Payment amount` can each take on 3,000 different values, uniformly distributed between 1 and 3000. For simplicity, we only consider integer values for payments (that is, no floats or decimal data types).

Payments are equally likely to be made on any of the 365 days of the year.

In general all database pages are 4,000 bytes with an average 75% fill factor. The only exception is if a single index data entry will take more than 75% of a page, then the database will fill as much space in that page as is required to keep that data entry completely in that leaf page. Also the root might have any fill factor.

Rids are 10 bytes each and internal page pointers are 6 bytes each. A data entry is NOT split across multiple pages.

You can assume that the root and intermediate nodes of an index are always in the memory (so you need not account for them in your memory requirements).

You can also assume there are 50 free buffer pages available in any question where this information is required, but you can add a few extra ones for the sake of simplifying calculations.

For simplicity, you can assume that the `Account` relation is split across 250 pages, the `Card` relation has 50 pages, and `Payment` has 1000 pages. (Feel free to calculate the actual estimated page numbers.)

Furthermore, you can assume an unclustered, type II B+-tree for `Payment` on `paydate` attribute. This index has 365 index leaf pages (in fact, each data entry is on a different leaf page)

Ex. 3 — Index Sizes (10 points)

Consider now a further the following further indirect unclustered, type II B+-tree for `Payment` on `amount` date.

For this index, indicate below values. Provide a simple calculation that shows how you have derived the values.:

1. *total number of data entries, the number of rids per data entry, and the size of a data entry*
2. *the number of leaf pages and the height of the tree*

For your own practice (and maybe also answering below questions with more confidence, you can also calculate the same information for the index on `paydate`.

Ex. 4 — Using indices (25 points)

One typical query checks the `Payment` table for payments above an amount `M` that were made on a particular date `D`.

```
SELECT *
FROM  Payment
WHERE paydate = D AND amount > M
```

1. Indicate the access costs (in number of pages retrieved leading to I/O) for this query in each of the scenarios for the case that D=2025-03-05 and M=2750. Provide a simple calculation that shows how you have derived the values.
   (a)(4 Points) you do not use any index.
   (b)(6 Points) you use the index on paydate (see above).
   (c)(6 Points) you use the index on amount (see above).
   (d)(4 Points) you use both indexes.
2. (0 Points) Calculate this also for general values of D and M.
3. (5 Points) What would be the access costs if the index on `paydate` was clustered? Give a short calculation that explains your answer.

Ex. 5 — Joins (15 Points)

Assume now a join between `Account` and `Payment` (WHERE Account.accountid = Payment.srcaccountid). Assume an unclustered type II index on `accountid` of `Account`.

Indicate

1.(0 points) the number of output tuples.

2.(5 Points) the estimated number of I/O when using an index nested loop join.

3.(5 Points) the estimated number of I/O when using block nested loop join and `Account` is the outer relation.

4.(5 Points) the estimated number of I/O when using sort merge join. Assume relations are not already sorted on the join attribute.
Hint:- Remember we used a technique in class to reduce the cost of merge-join so that it is less than the sum of the individual costs of merge sort and join.

Hints & Guidelines Query Evaluation

Note that these guidelines have been developed for query evaluation questions over the years. Specific points might not be relevant for the particular assignment of a given year.

- You can use the information (leaf pages, data entries, etc.) computed in one Exercise, in the following exercises.
- You can leave fractions as it is, if you would like to (as in number of data entries per page, records per page, etc.). However, keep in mind that there are some steps that cannot produce fractions. For example, if you have a specific block nested join step in your execution plan, its cost cannot be 24.6 pages, it has to be 27 pages - because that is the unit in which we do I/O (same with memory buffers).
- Minor rounding errors due to approximating data entries per page, records per page, etc., are acceptable. In general look at the magnitudes of the numbers you are rounding and how big a number you are multiplying it with later to see how significant the impact is. (for example, rounding 1.5 to 2 and rounding 1000.5 to 1001 introduces very different magnitude shifts to the numbers with which they are multiplied later). At the end of the day, remember that our objective is to have a reasonably good computational methodology to help us choose between multiple possible execution plans based on their costs.
- Partial points are given to steps and methodology even if you making errors doing math. But try to use a calculator to do your math to avoid making mistakes.

Appendix: Graph Data

The following contains the data for the example graph in form of a CREATE statement that allows you to load the data into Neo4j.

Before you do so, we can delete all existing data with the following command (useful to clean up and start from scratch).

```
//WARNING !! this command will delete everything in your graph database
MATCH(n) DETACH DELETE n;
```

```
// Create all the data required for the assignment in one shot !!
CREATE
  (s1:Student {name:'Mike', city:'Montreal'}) ,(s2:Student {name:'Kaiwen', city:'Montreal'})
 ,(s3:Student {name:'Fatemeh'}) ,(s4:Student {name:'Sally', city:'Montreal'})
 ,(s5:Student {name:'John'}) ,(s6:Student {name:'Dora', city:'Ottawa'})
 ,(s7:Student {name:'Benny', city:'Ottawa'}) ,(s8:Student {name:'Etienne', city:'Toronto'})
 ,(s9:Student {name:'Mara'})
 ,(u1:University {name:'McGill'}) ,(u2:University {name:'University of Ottawa'})
 ,(u3:University {name:'University of Toronto'})
 ,(f1:Faculty   {name:'Science'}) ,(f2:Faculty   {name:'Arts'})
 ,(f3:Faculty   {name:'Engineering'}), (f4:Faculty   {name:'Science'})
 ,(f5:Faculty   {name:'Arts'}), (f6:Faculty   {name:'Engineering'})
 ,(d1:Department   {name:'Computer Science'}) ,(d2:Department   {name:'Geography'})
 ,(c1:Course {name:'COMP202', title:'Fundamentals of Programming'})
 ,(c2:Course {name:'COMP421', title:'Database Systems'})
 ,(c3:Course {name:'COMP360', title:'Algorithm Design'})
 ,(c4:Course {name:'GEOG301', title:'Geography of Nunavut'})
 ,(c5:Course  {name:'GEOG417', title:'Urban Geography'})
 ,(t1:Type  {name:'200-level'}) ,(t2:Type  {name:'300-level'}),(t3:Type  {name:'400-level'})
 ,(t4:Type  {name:'applied'}),(t5:Type  {name:'foundational'})
 ,(s1)-[:FRIEND_OF]->(s2) ,(s2)-[:FRIEND_OF]->(s5)
 ,(s4)-[:FRIEND_OF]->(s5) ,(s4)-[:FRIEND_OF]->(s6)
 ,(s7)-[:FRIEND_OF]->(s6) ,(s7)-[:FRIEND_OF]->(s8)
 ,(s3)-[:FRIEND_OF]->(s9) , (s9)-[:FRIEND_OF]->(s5)
 ,(s2)-[:ROOMMATES]->(s4), (s8)-[:ROOMMATES]->(s7)
 ,(f1)-[:BELONGS_TO]->(u1) ,(f2)-[:BELONGS_TO]->(u1) ,(f3)-[:BELONGS_TO]->(u1)
 ,(f4)-[:BELONGS_TO]->(u2), (f5)-[:BELONGS_TO]->(u2) ,(f6)-[:BELONGS_TO]->(u3)
,(d1)-[:BELONGS_TO]->(f1) ,(d2)-[:BELONGS_TO]->(f2)
 ,(s1)-[:STUDY_AT {grad_year:2021}]->(u1) ,(s2)-[:STUDY_AT]->(u2)
 ,(s6)-[:STUDY_AT {grad_year:2022}]->(u3) ,(s3)-[:STUDY_AT]->(u3)
 ,(s4)-[:STUDY_AT {grad_year:2024}]->(u3) ,(s7)-[:STUDY_AT]->(u3)
 ,(s1)-[:BELONGS_TO]->(f1) ,(s9)-[:BELONGS_TO]->(f1)
 ,(s4)-[:BELONGS_TO]->(f2) ,(s5)-[:BELONGS_TO]->(f2),(s6)-[:BELONGS_TO]->(f2)
 ,(s8)-[: BELONGS_TO]->(f2),(s2)-[:BELONGS_TO]->(f3)
 ,(s3)-[:TAKES]->(c1),(s5)-[:TAKES]->(c2) ,(s5)-[:TAKES]->(c3)
 ,(s3)-[:TAKES]->(c3) ,(s3)-[:TAKES]->(c5) ,(s9)-[:TAKES]->(c4)
 ,(s1)-[:BELONGS_TO]->(d1) ,(s5)-[:BELONGS_TO]->(d2)
 ,(c1)-[:IS_OF_TYPE]->(t1), (c1)-[:IS_OF_TYPE]->(t4) ,(c1)-[:IS_OF_TYPE]->(t5)
 ,(c2)-[:IS_OF_TYPE]->(t3) ,(c2)-[:IS_OF_TYPE]->(t4),(c3)-[:IS_OF_TYPE]->(t2)
 ,(c3)-[:IS_OF_TYPE]->(t5),(c4)-[:IS_OF_TYPE]->(t4) ,(c4)-[:IS_OF_TYPE]->(t3)
 ,(c5)-[:IS_OF_TYPE]->(t3)
,(c1)-[:GIVEN_AT]->(u1), (c2)-[:GIVEN_AT]->(u1),(c3)-[:GIVEN_AT]->(u1)
, (c4)-[:GIVEN_AT]->(u1),(c5)-[:GIVEN_AT]->(u1)
;

//See your entire graph (Turn on Auto complete in the display view after running the query)
//MATCH(n) RETURN n;
```